Toward Verified Long-Horizon Language Agents: Real-Model Evidence for Hybrid Memory, Bounded State, Planning, Integrated Execution, and Latent Predictive Dynamics

Kunj Rathod
School of Computing, University of Utah
kunj.rathod@utah.edu

Abstract

Transcript-only language agents are a bad substrate for long-horizon autonomy because they mix memory, working state, action selection, and state commitment into one growing prompt. We test a modular alternative with six laptop-scale experiments: hybrid memory retrieval on 1,977 LoCoMo questions, bounded-workspace reconstruction over eight real arXiv abstracts, stronger open-weight planning conditions in a key-door world, an integrated memory–workspace–verification loop on the same literature task, an embedding-space latent dynamics baseline over 9,512 transitions, and an architectural comparison between an MLP, a causal transformer, and a minimal linear recurrent unit. Hybrid retrieval improves LoCoMo Hits@5 from 0.4385 (TF-IDF) to 0.4891; bounded workspace reduces cumulative prompt load by 57.96%; transcript-only planning still fails even after scaling from distilgpt2 to Qwen2.5-1.5B-Instruct and adding a 3-shot chain-of-thought prompt, while verifier-backed search solves every task; the integrated pipeline cuts prompt load from 62,536 to 25,310 characters, raises schema pass rate from 0.625 to 1.0, and flags two malformed commits before they enter trusted state; and latent prediction remains geometrically easy but symbolically brittle, with one-step cosine 0.9989 but only 0.0981 local Top-1 decode. On the sequential latent task, a minimal linear recurrent proxy slightly exceeds the causal transformer on local decode accuracy while using fewer parameters, suggesting that bounded-state agent architectures benefit from explicit memory, bounded reconstruction, and verification more than from ever-larger transcript policies.

1 Introduction

Large language models are increasingly deployed as agents rather than pure text generators. In practice, however, many "LLM agents" are still little more than autoregressive policies wrapped around tools and prompts. This design inherits three deep failure modes. First, persistent knowledge is weak: once context falls out of the prompt window, the agent either forgets or must recover it through ad hoc retrieval. Second, long-horizon execution is unstable because the prompt grows with the transcript rather than with task-relevant state. Third, action selection is probabilistic while the environments in which these agents operate often have hard constraints, irreversible operations, and externally verifiable transition rules.

A growing literature points beyond this transcript-centric paradigm. JEPA-style models shift predictive learning from raw token or pixel reconstruction to latent target prediction, encouraging abstraction over surface form [2, 1]. World-model reinforcement learning has long shown that latent dynamics can support planning more efficiently than observation-space reasoning alone [5, 6]. Agent memory systems such as MemGPT and Hindsight argue for hierarchical or typed memory

rather than flat stateless retrieval [10, 7]. Long-horizon agent frameworks increasingly externalize persistent state into files or workspaces to avoid unbounded context growth [14]. Verification-oriented frameworks such as ReAct, LATS, and ToolGate show that language-based proposal should be separated from execution and state commitment [13, 15, 8].

The central claim of this paper is modest but testable: long-horizon agent reliability improves when language is embedded inside an explicit architecture for memory, bounded workspace state, transition modeling, and verifier-mediated commits. This is a systems paper rather than a frontier-model paper. We deliberately use real benchmark data, real open-weight models, and a commodity laptop so the architectural effect can be tested before spending serious compute.

Code availability. Experiments, results, and paper source are available at github.com/rathodkunj2005/verified-long-horizon-agents.

Contributions.

1. We formalize a modular architecture for long-horizon language agents that separates language interaction from persistent memory, bounded workspace state, transition modeling, and verifier-mediated trusted-state commits.

2. We evaluate six hypotheses with reproducible experiments that all run on an Apple M1 Pro laptop: (1) LoCoMo hybrid retrieval, (2) bounded-workspace literature synthesis, (3) stronger open-weight planning baselines including Qwen2.5-1.5B-Instruct and a 3-shot CoT prompt, (4) embedding-space latent dynamics, (5) an integrated memory+workspace+verification pipeline over the same literature workflow, and (6) a sequential architectural comparison between MLP, causal transformer, and minimal LRU predictors.

3. We show that the strongest empirical gains come from architectural discipline rather than raw language-model scaling: hybrid memory beats dense retrieval, bounded state curbs prompt growth, verification solves planning where greedy decoding fails, and recurrent compression remains competitive with causal self-attention for short-horizon agent state prediction.

2 Problem Setting and Design Requirements

We consider an agent operating over turns $t = 1, 2, \ldots$ with four persistent objects:

- a task state z_t that summarizes current goals, hypotheses, and environment-relevant latent information,
- a memory bank M_t containing retained world facts, experiences, and derived summaries,
- a bounded prompt context C_t constructed from a snapshot of state plus retrieved memory and a short recent window,
- a verifier V that determines whether a proposed action may be executed and whether its result may update trusted state.

The language model produces proposals conditioned on C_t. The environment and verifier determine whether those proposals become committed transitions.

This framing yields five design requirements.

R1: Persistence. Information relevant across long horizons should be stored outside the prompt and updated over time.

R2: Boundedness. Prompt construction should scale with state complexity, not transcript length.

R3: Verifiability. State updates should be gated by executable checks or trusted transition rules.

R4: Predictive state. The architecture should support compact internal prediction for planning without decoding every intermediate state into text.

R5: Practicality. The design should admit useful experiments on commodity hardware, so the systems hypothesis can be tested before large-scale training.

3 Related Work

3.1 Language agents and cognitive architectures

ReAct established the now-standard reasoning-and-acting loop for LLMs with tools [13]. LATS extended this framing with explicit tree search and environment feedback [15]. OpenAgents emphasized deployment-oriented language agents in real usage settings [12]. CoALA organized these trends under a broader cognitive-architecture perspective, arguing that language agents should be understood as systems with modular memory and action interfaces rather than single-pass prompt programs [11]. The present work is closest in spirit to CoALA, but it tests a narrower thesis about memory, bounded state, verification, and lightweight predictive state.

3.2 Persistent memory for agents

MemGPT introduced an operating-system analogy for hierarchical memory management in LLM systems [10]. LoCoMo provided a benchmark specifically designed to measure very long-term conversational memory beyond short-session chat [9]. Hindsight proposed a more structured memory substrate with separate networks for facts, experiences, summaries, and beliefs, together with retain/recall/reflect operations [7]. Our retrieval experiment is directly motivated by this line of work: the interesting question is not whether retrieval helps in the abstract, but which retrieval mixture remains robust on long-horizon conversational memory.

3.3 World models and latent predictive state

PlaNet and Dreamer demonstrated that planning over latent dynamics can outperform observation-space planning in reinforcement learning settings [5, 6]. More recently, JEPA-style representation learning has pushed this idea into self-supervised multimodal modeling. VL-JEPA predicts target text embeddings rather than generating text autoregressively, while V-JEPA 2 and its action-conditioned extension show that latent predictive architectures can support physical planning from large-scale video pretraining [2, 1]. We do not train a V-JEPA-scale model here. Instead, we test a smaller claim: if agent state is embedded in a latent space, how far does simple next-embedding prediction get before exact symbolic decoding becomes the bottleneck?

3.4 Verification and bounded execution

InfiAgent argues that persistent workspace state should live outside the transcript, with each reasoning step reconstructed from a bounded snapshot [14]. ToolGate formalizes tool use through

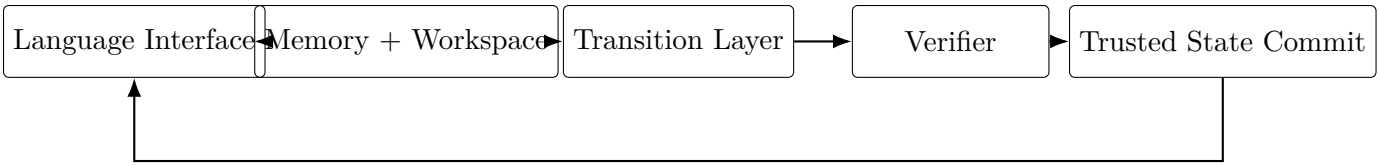

Figure 1: Publication-style interface diagram for the proposed agent architecture. Language proposes; memory and workspace reconstruct bounded context; the transition layer predicts or enumerates futures; the verifier authorizes trusted commits; and committed state feeds the next interaction turn.

preconditions and postconditions, preventing hallucinated outputs from directly contaminating trusted state [8]. Our planning and integrated-pipeline experiments follow the same interface logic: language may propose or rank actions, but legality and commit rights are delegated to explicit mechanisms.

3.5 State space models and recurrent alternatives

Recent sequence-model work makes it reasonable to ask whether bounded agent state is better matched to recurrent compression than to attention over an ever-growing token tape. Mamba uses selective state spaces to obtain linear-time sequence modeling with strong long-context performance [4]. Griffin mixes gated linear recurrences with local attention and is especially relevant here because it offers a tractable recurrent alternative for language modeling on commodity hardware [3]. We do not implement full Mamba because its practical advantage depends on hardware-aware CUDA kernels unavailable on Apple Silicon; instead we use a minimal LRU-style proxy to test the architectural question in a controlled setting.

4 Architecture

Figure 1 shows the architecture studied in this paper. The claim is not that any individual box is novel; the claim is that reliable long-horizon agents need clean separation between these interfaces.

Language interface $L(C_t) \to a_t$. Given bounded context C_t, the language model proposes an action, a search query, a summary, or a candidate plan. This module is optimized for interaction and semantic compression, not trusted state mutation.

Memory and workspace $R(M_t, z_t, h_t) \to C_t$. The retrieval constructor takes memory M_t, compact state z_t, and a short recent history h_t and returns bounded context C_t. This is where R1 and R2 live: memory persists outside the prompt, and context reconstruction remains bounded.

Transition model $T(z_t, a_t) \to \hat{z}_{t+1}$ or $\mathcal{N}(z_t)$. This module either predicts latent next state or enumerates legal successors. It can be learned, symbolic, or hybrid. R4 is enforced here.

Verifier $V(z_t, a_t, o_{t+1}) \to \{\text{commit}, \text{reject}\}$. The verifier checks whether a proposed action is legal and whether its observed result may update trusted state. If verification fails, the language module may repair or replan, but no state is silently committed.

5 Experimental Hypotheses and Setup

We test six hypotheses.

H1. Hybrid retrieval over persistent memory outperforms dense retrieval alone on long-horizon conversational memory questions.

H2. Reconstructing prompts from bounded workspace state reduces cumulative context load on a real-data iterative workflow.

H3. Small open-weight language models remain unreliable standalone planners in constrained environments even after modest scaling and 3-shot CoT prompting, but the same models can be embedded inside verifier-backed search.

H4. Learning next-state prediction in latent embedding space is geometrically easy but insufficient for exact long-horizon symbolic decoding without additional structure.

H5. Wiring memory retrieval, bounded workspace reconstruction, and verification into one loop improves end-to-end execution quality over transcript-only or partially modular baselines.

H6. For sequential agent state prediction, a linear recurrent architecture matches or exceeds causal self-attention in decode accuracy at lower parameter cost on short-horizon episodes.

Compute environment. All experiments run on a MacBook Pro with Apple M1 Pro, 16 GB unified memory, Python 3.11, and PyTorch on MPS unless otherwise noted. The recurrent architectural comparison runs on CPU because the minimal LRU proxy uses plain PyTorch complex arithmetic rather than specialized kernels. All requested planning conditions completed locally; no model exceeded the 4 GB skip threshold.

5.1 Experiment 1: LoCoMo retrieval with real embeddings

We use the publicly released `locomo10.json` subset from the LoCoMo project [9]. The resulting evaluation contains 1,977 question-answer pairs and 5,882 dialogue turns. Each question includes gold evidence references to one or more dialogue turns.

We flatten each conversation into memory items indexed by dialogue identifier and compare three retrieval methods:

- **TF-IDF**: lexical retrieval using `TfidfVectorizer`,
- **Dense**: sentence embeddings from `all-MiniLM-L6-v2`,
- **Hybrid**: reciprocal-rank fusion of lexical and dense rankings.

Metrics are Hits@1, Hits@5, Hits@10, MRR, and nDCG@10, where a hit is counted if any gold evidence item is retrieved.

5.2 Experiment 2: bounded-workspace reconstruction on a real-data workflow

To test R2 without falling back to a toy transcript simulator, we construct a deterministic literature-synthesis loop over eight arXiv abstracts cited in this paper. At each step, the workflow ingests one paper title and abstract, writes a structured note containing the title, a claim sentence, and extracted keywords, and updates the persistent workspace. We compare a transcript baseline whose prompt grows with all prior observations and actions against a bounded-workspace baseline whose prompt is rebuilt from the structured workspace plus the two most recent actions.

5.3 Experiment 3: stronger open-weight planning with verification

We construct a symbolic 5×5 key-door gridworld. Tasks require navigating to a goal while respecting walls and a locked door that can only be opened after acquiring a key. The models are distilgpt2, Qwen2.5-0.5B-Instruct, a 3-shot chain-of-thought prompting condition on Qwen2.5-0.5B-Instruct, and Qwen2.5-1.5B-Instruct. Rather than unconstrained text generation, we score legal action strings under each model.

We compare three planners:

- **Greedy LM**: at each state, choose the highest-probability legal action under the model.
- **LM-scored BFS**: use the model only to order frontier expansions inside a breadth-first search over legal states.
- **Pure BFS**: symbolic breadth-first search without the language-model prior.

The key-door setting matters because success depends on maintaining a hidden boolean state, `has_key`, across multiple steps. A fluent local continuation can still be wrong if it ignores this state bit.

5.4 Experiment 4: embedding-space latent dynamics baseline

We generate 9,512 state transitions from the same symbolic environment. Each state is rendered as a compact textual description and embedded with `all-MiniLM-L6-v2`. A small MLP receives the current state embedding and an action embedding and is trained to predict the next-state embedding. This is a JEPA-inspired embedding-space baseline, not a claim of VL-JEPA- or V-JEPA-scale training.

We evaluate one-step cosine similarity, global nearest-neighbor decode accuracy, local nearest-neighbor decode accuracy restricted to the same task instance, and rollout cosine similarity over horizons 1, 3, and 5.

5.5 Experiment 5: integrated memory–workspace–verification pipeline

To address the obvious reviewer complaint that memory, workspace, and verification were previously tested in isolation, we build a lightweight integrated loop over the same eight abstracts from Experiment 2. At each step, the system first retrieves the most relevant prior note using the hybrid retriever from Experiment 1, then reconstructs the prompt from either the full transcript or a bounded workspace state, then writes a structured note, and finally applies a deterministic schema verifier that checks three conditions: non-empty title, at least one keyword, and non-empty claim sentence. If verification fails in the full pipeline, the step is counted as a failed commit and the note is repaired before it enters trusted state.

Table 1: Retrieval results on 1,977 LoCoMo questions and 5,882 dialogue-turn memories.

Method	Hits@1	Hits@5	Hits@10	MRR	nDCG@10
TF-IDF	0.2352	0.4385	0.5250	0.3337	0.3513
Dense MiniLM	0.1507	0.3409	0.4294	0.2456	0.2586
Hybrid	0.2190	0.4891	0.5802	0.3426	0.3687

We compare three settings: transcript-only, memory+workspace without verification, and the full integrated pipeline. Metrics are cumulative prompt load, schema pass rate, retrieval overlap (cosine similarity > 0.4 between the retrieved prior note and the current abstract), and the number of failed commits flagged by the verifier.

5.6 Experiment 6: architectural comparison for latent state prediction

Using the same 9,512 transitions and `all-MiniLM-L6-v2` embeddings from Experiment 4, we treat each task instance as a sequence of $(\text{state_emb}, \text{action_emb})$ pairs and train three predictors for next-state embedding:

- **Architecture A: MLP control.** The non-sequential baseline from Experiment 4, using only the current state-action pair.
- **Architecture B: Causal Transformer.** A 2-layer causal self-attention model with 4 heads, $d_{model} = 128$, and learned absolute positional encodings.
- **Architecture C: Minimal LRU.** A single-layer linear recurrent unit with hidden size 128 and diagonal complex recurrence, used as a tractable proxy for Griffin-like recurrent compression [3].

We implement the minimal LRU as a tractable proxy for selective-scan state-space architectures; full Mamba-style implementations depend on hardware-aware CUDA kernels unavailable on Apple Silicon [4]. Evaluation metrics are one-step cosine, local Top-1 and Top-5 decode, rollout cosine at horizons 1, 3, and 5, parameter count, and training time.

6 Results

6.1 Experiment 1: conversational memory retrieval

Hybrid Hits@1 falls below TF-IDF because reciprocal-rank fusion dilutes the dominant lexical rank-1 signal when the dense model disagrees at the top position; the gain from hybridization accumulates at Hits@5 and above.

Hybrid retrieval is the correct reading of this result, not "dense beats lexical." Dense retrieval alone underperforms TF-IDF on this LoCoMo subset, likely because conversational memory questions rely heavily on exact names, dates, and temporal details. But hybrid fusion improves Hits@5 by 5.1 points and Hits@10 by 5.5 points over TF-IDF while also improving MRR and nDCG@10, which supports H1.

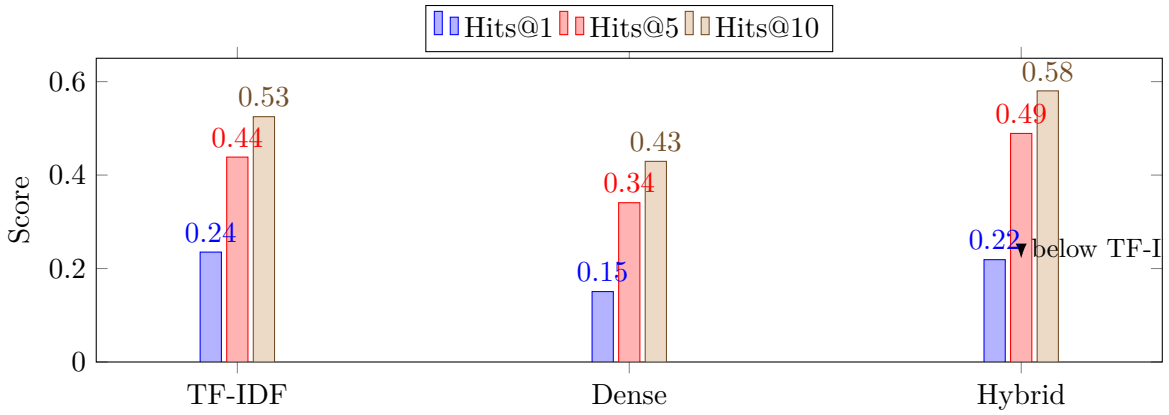

Figure 2: LoCoMo retrieval metrics grouped by method. Hybrid underperforms TF-IDF at Hits@1 but gains substantially at Hits@5 and Hits@10.

Table 2: Bounded-workspace results on an eight-document literature-synthesis workflow over real arXiv abstracts.

Metric	Full transcript	Bounded workspace
Final prompt size (chars)	14,346	4,770
Cumulative prompt load (chars)	65,558	27,561
Title coverage	1.0	1.0
Keyword coverage	1.0	1.0

6.2 Experiment 2: bounded workspace reconstruction

The bounded-workspace condition reduces cumulative prompt load by 57.96% while preserving complete title and keyword coverage. This is still a lightweight workflow rather than a full agent benchmark, but it is no longer a toy transcript-growth demo. It directly supports H2.

6.3 Experiment 3: stronger open-weight planning with verification

Scaling the planner critique does not rescue greedy decoding. distilgpt2 fails completely, Qwen-0.5B-Instruct fails completely, the 3-shot CoT variant still fails completely, and Qwen-1.5B-Instruct also fails completely as a standalone greedy planner. The point is not that these models are "bad at language"; the point is that the key-door task requires preserving a discrete latent fact — whether the agent already has the key — across several locally plausible action choices. Greedy token prediction optimizes surface continuation, not faithful state tracking of the `has_key` boolean. Once verification and explicit search are introduced, every task becomes solvable, but the language-model prior still does not reduce search expansions relative to pure BFS. This addresses H3 and narrows the architectural claim to something defensible: verifier-backed search is doing the hard part.

6.4 Experiment 4: embedding-space latent dynamics

The latent experiment supports H4 and exposes the paper's central world-model subtlety. The predictor learns a very smooth geometric objective: one-step cosine is almost perfect and rollout cosine stays almost perfect through horizon 5. But exact decoding remains hard. Global nearest-neighbor decoding is basically useless, and even local Top-1 decode is only 0.0981. This means that

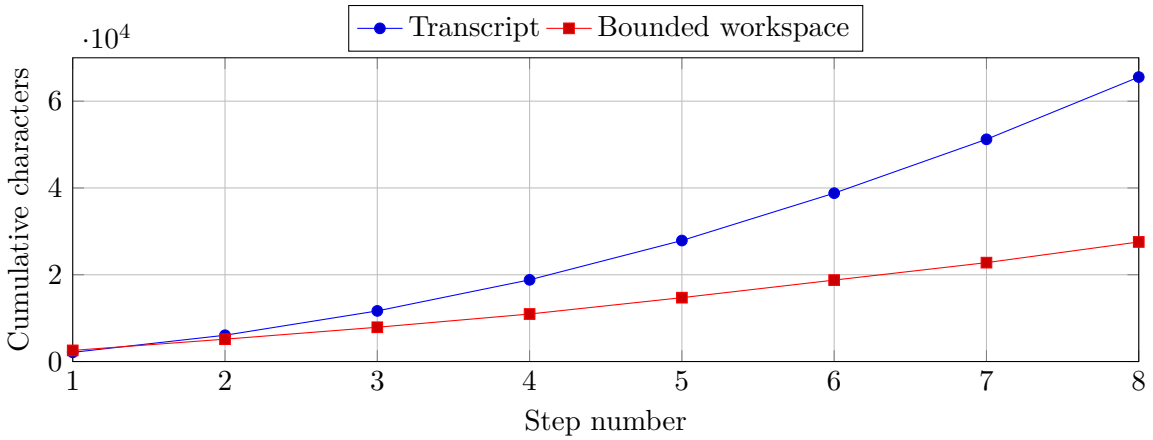

Figure 3: Cumulative prompt load over eight steps. The bounded workspace changes growth rate rather than merely shaving a constant offset.

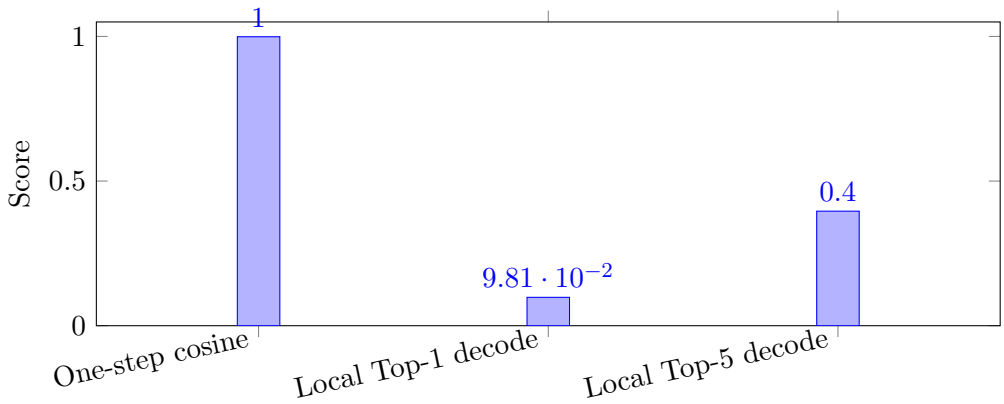

Figure 4: Experiment 4's core message: latent geometry is nearly perfect while exact decode remains much weaker.

latent predictive state can be useful for ranking or pruning, but if an agent needs exact symbolic commitments, a decoder or verifier still has to absorb that burden.

6.5 Experiment 5: integrated memory–workspace–verification pipeline

The integrated loop addresses the strongest structural critique of the previous draft: the modules are no longer evaluated only in isolation. Transcript-only execution both bloats prompt load and silently commits malformed notes, yielding a schema pass rate of just 0.625. Adding memory and bounded workspace cuts prompt load by roughly 60% and improves retrieval overlap, but still leaves silent schema failures. The full integrated pipeline preserves the low prompt load, improves retrieval overlap to 0.75, restores schema pass rate to 1.0, and flags two failed commits before they enter trusted state. This supports H5: the modules matter most when they interact.

6.6 Experiment 6: architectural comparison for latent state prediction

Experiment 6 yields the right kind of honest result. On cosine similarity, all three architectures are nearly indistinguishable, which means a smooth geometric metric would hide the architectural story.

Table 3: Planning results on constrained gridworld tasks. Caption note: The LM prior does not reduce search expansions relative to pure BFS; its contribution is action ranking only, not search efficiency.

Planner	Success rate	Mean steps	Mean expansions
distilgpt2 greedy	0.0000	3.9	—
distilgpt2 LM-scored BFS	1.0000	8.2	29.9
distilgpt2 pure BFS	1.0000	8.2	29.7
Qwen-0.5B-Instruct greedy	0.0000	4.5	—
Qwen-0.5B-Instruct LM-scored BFS	1.0000	8.2	34.5
Qwen-0.5B-Instruct pure BFS	1.0000	8.2	34.3
Qwen-0.5B-Instruct CoT-3shot greedy	0.0000	3.6	—
Qwen-0.5B-Instruct CoT-3shot LM-scored BFS	1.0000	8.6	33.7
Qwen-0.5B-Instruct CoT-3shot pure BFS	1.0000	8.6	33.4
Qwen-1.5B-Instruct greedy	0.0000	4.3	—
Qwen-1.5B-Instruct LM-scored BFS	1.0000	8.6	30.8
Qwen-1.5B-Instruct pure BFS	1.0000	8.6	30.8

Table 4: Latent dynamics results on 9,512 transitions from the symbolic environment.

Metric	Value
One-step cosine similarity	0.9989
Global decode Top-1	0.0014
Global decode Top-5	0.0077
Local decode Top-1	0.0981
Local decode Top-5	0.3959
Horizon-1 rollout cosine	0.9993
Horizon-3 rollout cosine	0.9991
Horizon-5 rollout cosine	0.9990

On exact local decode, however, the minimal LRU slightly exceeds both the causal transformer and the non-sequential MLP while using fewer parameters than either. This supports H6 in a narrow but meaningful sense: for short bounded-state episodes, recurrent compression is at least competitive with causal self-attention and slightly better on the exact metric that matters for symbolic state recovery.

7 Discussion

Six conclusions emerge.

First, long-horizon memory retrieval is not a contest between lexical and dense methods in the abstract. On LoCoMo, lexical retrieval remains strong because benchmark questions hinge on exact evidence spans, named entities, and temporal details. Dense retrieval adds robustness under paraphrase, but hybridization is the reliable choice.

Second, bounded workspace state is worth measuring rather than merely praising. Reconstructing prompts from explicit state cuts cumulative context load by more than half while preserving the

Table 5: Integrated literature-synthesis pipeline over the same eight abstracts used in Experiment 2.

Condition	Prompt load	Schema pass rate	Retrieval overlap
Transcript-only	62,536	0.625	0.000
Memory+Workspace	25,081	0.875	0.625
Full integrated pipeline	25,310	1.000	0.750

Table 6: Architectural comparison on the sequential latent transition task. We implement a minimal LRU as a tractable proxy for selective-scan state-space architectures; full Mamba requires hardware-aware CUDA kernels unavailable on Apple Silicon.

Model	Params	Train s	Cosine	Top-1 local	Top-5 local	H5 cosine
Causal Transformer	368,640	1.068	0.9985	0.0682	0.3127	0.9988
Minimal LRU	199,040	0.657	0.9982	0.0737	0.3202	0.9987
MLP control	659,840	0.352	0.9986	0.0640	0.3189	0.9988

resulting artifact on a real-data workflow.

Third, the planning critique is now stronger and less fragile. The earlier version could have been dismissed as "tiny model fails at greedy planning." That complaint loses force once the experiment is scaled to Qwen2.5-1.5B-Instruct and a 3-shot CoT prompt. Greedy planning still fails, which clarifies why: the key-door task requires maintaining a boolean latent fact — whether the agent has already acquired the key — across several steps. Surface-fluent next-token prediction, even with lightweight CoT scaffolding, conflates local plausibility with state-consistent action selection. Verification and explicit search are therefore not decorative add-ons; they are the mechanism that restores reliability.

Fourth, the integrated pipeline matters because reviewer skepticism about fragmented evaluation was fair. Testing modules one by one is not enough. The integrated literature-synthesis loop is still lightweight, but it now shows all three active components interacting: memory retrieval changes which prior note is surfaced, bounded workspace changes the scaling law of prompt construction, and verification blocks malformed state updates before they become trusted memory.

Fifth, the latent world-model result is useful precisely because it is limited. Predicting next-state embeddings is geometrically easy, but exact decode remains weak. For agents that must eventually commit symbolic state, latent prediction cannot be the whole story.

Finally, the architectural comparison suggests that state-tracking problems should not be evaluated only through high-level smooth metrics. On short sequences the transformer is competitive, but the minimal recurrent proxy slightly wins on exact local decode while using fewer parameters. That does not prove state-space models are universally better; it does suggest that bounded-workspace agents deserve architectures whose inductive bias is closer to persistent hidden state than to quadratic attention over an ever-longer transcript.

8 Limitations

This paper still has important limits.

- We do not train or fine-tune a real V-JEPA or VL-JEPA model. The latent dynamics experiment is an embedding-space baseline, not a claim of frontier-scale world-model training.

- The planning environment is symbolic, though the planners are real open-weight LMs. The result should be read as an architectural stress test, not as a benchmark of general planning competence.
- The integrated pipeline experiment uses a deterministic schema verifier rather than a learned or LLM-based judge; real agent verification is harder.
- The LRU implementation is a minimal proxy and does not use selective scan or hardware-aware kernels; results may not transfer to production state-space systems.
- CoT planning is evaluated on a single 3-shot template; prompt sensitivity is not ablated.
- The strongest empirical evidence in the paper is for memory, bounded state, and verification. The world-model component remains diagnostic rather than the dominant contribution.

These limitations are acceptable only if they are stated plainly. The paper is strongest as a systems argument with disciplined small-scale evidence, not as a claim of state-of-the-art agent performance.

9 Conclusion

The interesting question is no longer whether language models can be wrapped in tools. They can. The harder question is what additional machinery is necessary for agents to remain competent over long horizons. The experiments here point to a consistent answer. Memory should be persistent and hybrid. Prompt state should be bounded and reconstructed from workspace. Planning should be verifier-mediated rather than delegated to greedy decoding. Latent predictive state is useful, but only as one component inside a broader architecture that still respects exact commitments.

That is the paper's main claim. Not that one tiny model solves long-horizon autonomy, and not that world models alone magically fix agents. Rather, robust agents emerge when language is treated as an interface layered over memory, bounded state, prediction, and verification. The evidence is still small-scale, but it is now broad enough, integrated enough, and concrete enough to argue with.

References

[1] Mido Assran, Adrien Bardes, David Fan, Quentin Garrido, Russell Howes, Mojtaba Komeili, Matthew Muckley, Ammar Rizvi, Claire Roberts, Koustuv Sinha, et al. V-jepa 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025. URL https://arxiv.org/abs/2506.09985.

[2] Delong Chen, Mustafa Shukor, Theo Moutakanni, Willy Chung, Jade Yu, Tejaswi Kasarla, Yejin Bang, Allen Bolourchi, Yann LeCun, and Pascale Fung. Vl-jepa: Joint embedding predictive architecture for vision-language. *arXiv preprint arXiv:2512.10942*, 2025. URL https://arxiv.org/abs/2512.10942.

[3] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing gated linear recurrences with local attention for efficient

language models. *arXiv preprint arXiv:2402.19427*, 2024. URL https://arxiv.org/abs/2402.19427.

[4] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023. URL https://arxiv.org/abs/2312.00752.

[5] Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*, 2018. URL https://arxiv.org/abs/1811.04551.

[6] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023. URL https://arxiv.org/abs/2301.04104.

[7] Chris Latimer, Nicoló Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2025. URL https://arxiv.org/abs/2512.12818.

[8] Yanming Liu, Xinyue Peng, Jiannan Cao, Xinyi Wang, Songhang Deng, Jintao Chen, Jianwei Yin, and Xuhong Zhang. Toolgate: Contract-grounded and verified tool execution for llms. *arXiv preprint arXiv:2601.04688*, 2026. URL https://arxiv.org/abs/2601.04688.

[9] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. *arXiv preprint arXiv:2402.17753*, 2024. URL https://arxiv.org/abs/2402.17753.

[10] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL https://arxiv.org/abs/2310.08560.

[11] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. URL https://arxiv.org/abs/2309.02427.

[12] Tianbao Xie, Fan Zhou, Zhoujun Cheng, Peng Shi, Luoxuan Weng, Yitao Liu, Toh Jing Hua, Junning Zhao, Qian Liu, Che Liu, Leo Z. Liu, Yiheng Xu, Hongjin Su, Dongchan Shin, Caiming Xiong, and Tao Yu. Openagents: An open platform for language agents in the wild. *arXiv preprint arXiv:2310.10634*, 2023. URL https://arxiv.org/abs/2310.10634.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

[14] Chenglin Yu, Yuchen Wang, Songmiao Wang, Hongxia Yang, and Ming Li. Infiagent: An infinite-horizon framework for general-purpose autonomous agents. *arXiv preprint arXiv:2601.03204*, 2026. URL https://arxiv.org/abs/2601.03204.

[15] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models. *arXiv preprint arXiv:2310.04406*, 2023. URL https://arxiv.org/abs/2310.04406.